

EchoUART Controller

Hardware Design Specification

Language: SystemVerilog

1. Overview

EchoUART is a fully parameterizable UART loopback controller implemented in SystemVerilog. The design receives serial data on the RX line, buffers it in an internal synchronous FIFO, and retransmits each byte over the TX line. It is intended as a portfolio and educational reference for RTL design and UVM-based functional verification.

The design is structured as a single top-level module (uart_controller) that instantiates four submodules: a baud rate generator, a UART receiver FSM, a synchronous FIFO, and a UART transmitter FSM.

2. System Architecture

The data flow through the system is strictly unidirectional:

- i_rx_serial → uart_rx → fifo_sync → uart_tx → o_tx_serial

All submodules share a single clock domain derived from i_clk. The baud_rate_generator produces a 16x oversampling tick that is consumed by both uart_rx and uart_tx.

Submodule	Instance Name	Role
baud_rate_generator	u_baud_gen	Generates 16x oversampling baud tick
uart_rx	u_uart_rx	Receives and decodes serial UART frames
fifo_sync	u_fifo	Buffers received bytes for TX consumption
uart_tx	u_uart_tx	Encodes and transmits serial UART frames

3. Parameters

All parameters are defined at the uart_controller level and propagated to submodules. The following table describes each parameter:

Parameter	Type	Default	Description
CLK_FREQ	int	50_000_000	System clock frequency in Hz

Parameter	Type	Default	Description
BAUD_RATE	int	9_600	UART baud rate in bps
DATA_WIDTH	int	8	Number of data bits per UART frame
PARITY_EN	bit	1	Enable parity bit (0 = no parity)
PARITY_IS_EVEN	bit	1	1 = even parity; 0 = odd parity
FIFO_DEPTH	int	16	Depth of the synchronous FIFO (must be power of 2)

Note: $\text{CLK_PER_TICK} = \text{CLK_FREQ} / (\text{BAUD_RATE} \times 16)$ is derived internally by baud_rate_generator. If CLK_FREQ is not evenly divisible by (BAUD_RATE x 16), a fractional baud rate error will occur.

4. Port List

The following ports are exposed at the uart_controller top-level boundary:

Port Name	Direction	Width	Description
i_clk	input	1	System clock. All logic clocked on rising edge.
i_reset_n	input	1	Active-low asynchronous reset.
i_rx_serial	input	1	Serial UART RX input line.
o_tx_serial	output	1	Serial UART TX output line.
o_rx_error	output	1	Pulse: framing error detected on received frame.
o_parity_error	output	1	Pulse: parity mismatch detected on received frame.
o_tx_busy	output	1	High while TX FSM is transmitting a frame.
o_fifo_full	output	1	High when the internal FIFO is full.
o_fifo_empty	output	1	High when the internal FIFO is empty.

5. Submodule Specifications

5.1 baud_rate_generator

Generates a single-cycle pulse (o_baud_tick) at a rate of BAUD_RATE x 16 pulses per second. This 16x oversampling tick is used by both the RX and TX FSMs to time individual bit periods.

- An internal counter (tick_counter) increments each clock cycle.
- When tick_counter reaches CLK_PER_TICK - 1, o_baud_tick is asserted for exactly one clock cycle and the counter resets to 0.
- Counter width is automatically sized to $\lceil \log_2(\text{CLK_PER_TICK}) \rceil$ bits.
- o_baud_tick is registered, so it is glitch-free and synchronous with i_clk.

5.2 fifo_sync

A parameterizable synchronous single-clock FIFO. Depth and data width are configurable. The extra-bit pointer technique is used to distinguish full from empty without an occupancy counter.

Pointer scheme:

- Write pointer (w_ptr) and read pointer (r_ptr) are each Depth_log+1 bits wide, where Depth_log = $\lceil \log_2(\text{Depth}) \rceil$.
- empty condition: $r_ptr == w_ptr$ (all bits equal).
- full condition: $r_ptr[\text{Depth_log}] \neq w_ptr[\text{Depth_log}]$ AND $r_ptr[\text{Depth_log}-1:0] == w_ptr[\text{Depth_log}-1:0]$ (MSBs differ, lower bits equal).

Behavior:

- Write: On posedge clk, if w_en && !full, data is written to $\text{mem}[w_ptr[\text{Depth_log}-1:0]]$ and w_ptr increments.
- Read: On posedge clk, if r_en && !empty, r_data is updated from $\text{mem}[r_ptr[\text{Depth_log}-1:0]]$ and r_ptr increments. Read data is registered (1-cycle latency).
- Simultaneous read and write when neither full nor empty is supported.
- Write-while-full and read-while-empty are silently ignored.

5.3 uart_rx

A 5-state FSM that receives and decodes serial UART frames. The RX line is passed through a 2-flip-flop metastability synchronizer before any combinational or sequential logic uses it.

Metastability synchronizer:

- $i_rx_serial \rightarrow rx_meta \rightarrow rx_sync \rightarrow rx_sync_d1$
- start_detect is asserted when a falling edge is detected on the synchronized signal: $rx_sync_d1 \& \sim rx_sync$.

FSM states:

State	Entry Condition	Action
S_IDLE	Reset or end of frame	Wait for falling edge (start_detect).
S_START_BIT	start_detect asserted	Wait 8 baud ticks. Verify line is still low at mid-bit. If high, return to IDLE (glitch rejection).
S_DATA_BITS	Start bit verified	Sample rx_sync at tick 15 of each bit period. Shift into rx_shift_reg LSB-first for DATA_WIDTH bits.
S_PARITY_BIT	All data bits received (PARITY_EN = 1)	Sample rx_sync at tick 15. Compare with expected parity. Assert o_parity_error if mismatch.
S_STOP_BIT	Parity done or data done (PARITY_EN = 0)	Sample rx_sync at tick 15. If high: output o_data_valid and o_data_parallel. If low: assert o_rx_error (framing error).

Output behavior:

- o_data_valid, o_rx_error, and o_parity_error are all 1-cycle pulses (deasserted by default each cycle).
- o_parity_error and o_data_valid can both be asserted in the same frame if parity fails but stop bit is correct.
- A framing error (o_rx_error) suppresses o_data_valid; the byte is not forwarded.

5.4 uart_tx

A 5-state FSM that encodes and transmits UART frames serially. Transmission begins when i_tx_start is asserted for one cycle.

Frame format (LSB first):

- Start bit (0) → Data bits [0..DATA_WIDTH-1] → Parity bit (if PARITY_EN) → Stop bit (1)

FSM states:

State	Entry Condition	Action
S_IDLE	Reset or end of frame	o_tx_serial = 1, o_tx_busy = 0. On i_tx_start: latch data, compute parity, assert start bit (0) and o_tx_busy.
S_START_BIT	i_tx_start asserted	Hold start bit for 16 baud ticks. On exit: output tx_shift_reg[0] and shift right.
S_DATA_BITS	Start bit complete	Output one data bit per 16 baud ticks. After DATA_WIDTH-1 bits in this state (bit 0 was output in S_START_BIT exit): transition to parity or stop.
S_PARITY_BIT	All data bits sent, PARITY_EN=1	Output pre-computed parity_bit for 16 ticks, then transition to S_STOP_BIT.
S_STOP_BIT	Parity done or data done	Output stop bit (1) for 16 ticks. On exit: return to S_IDLE, deassert o_tx_busy.

Parity computation:

- parity_bit is computed at the transition from S_IDLE to S_START_BIT (i.e., on the same cycle as i_tx_start).
- Even parity: parity_bit = XOR of all data bits.
- Odd parity: parity_bit = XNOR of all data bits.

5.5 uart_controller (Top-Level)

The top-level module wires all submodules together and implements a 4-state TX control FSM that manages reading from the FIFO and feeding data to uart_tx.

RX path:

- uart_rx outputs o_data_valid and o_data_parallel when a valid frame is received.
- fifo_wen = rx_valid & ~fifo_full. Frames are silently dropped when FIFO is full.

TX control FSM:

State	Transition	Action
S_IDLE	!fifo_empty && !tx_busy	Assert fifo_ren, go to S_READ.
S_READ	Always (1 cycle)	Wait 1 cycle for registered FIFO read data to settle.
S_LOAD	Always (1 cycle)	Latch fifo_rdata into tx_data_latch. Assert tx_start for 1 cycle. Go to S_WAIT.
S_WAIT	!tx_busy	Wait until uart_tx deasserts o_tx_busy. Return to S_IDLE.

6. Timing Characteristics

The following timing values apply to the default configuration (CLK_FREQ = 50 MHz, BAUD_RATE = 9600):

Parameter	Formula	Default Value
Baud tick period	$\text{CLK_FREQ} / (\text{BAUD_RATE} \times 16)$	~326 clock cycles
Bit period	16 baud ticks	~5,208 clock cycles
Full frame (8N1)	10 bit periods	~52,083 clock cycles
Full frame (8E1)	11 bit periods	~57,291 clock cycles
RX start to valid	$8.5 + \text{DATA_WIDTH} + \text{parity} + 1 \text{ bits}$	~10.5 bit periods
TX latency (FIFO)	$\text{S_READ} + \text{S_LOAD} = 2 \text{ cycles overhead}$	2 clock cycles

7. Functional Behavior

7.1 Normal Operation

The normal loopback sequence for one byte is:

- 1. External source drives a valid UART frame on i_rx_serial.
- 2. uart_rx detects the falling edge start bit, samples 8 data bits and the parity bit, verifies the stop bit, and pulses o_data_valid.
- 3. fifo_wen is asserted (if FIFO is not full) and the byte is written into fifo_sync.
- 4. The TX controller detects !fifo_empty && !tx_busy, issues fifo_ren, latches the data, and asserts tx_start.
- 5. uart_tx transmits the byte serially on o_tx_serial with the same frame format.

7.2 Error Handling

- Framing error: If the stop bit is sampled low, o_rx_error is asserted for 1 cycle. The byte is NOT written to the FIFO.
- Parity error: If the parity bit does not match the expected value, o_parity_error is asserted for 1 cycle. The byte IS still written to the FIFO (controller does not filter on parity).
- FIFO full: If the FIFO is full when a valid byte arrives, fifo_wen is suppressed. The byte is silently dropped. o_fifo_full remains high.
- Glitch rejection: If the RX line is high when sampled at the mid-point of the start bit (8 ticks after falling edge), the FSM returns to S_IDLE without processing a frame.

7.3 Reset Behavior

- All FSMs return to their IDLE states.
- o_tx_serial is driven high (idle line state).
- FIFO read and write pointers reset to 0.
- All output pulses (o_data_valid, o_rx_error, o_parity_error) are deasserted.

8. Verification Summary

The design is verified using a UVM testbench. The following table summarizes the verification environment components:

Component	Description
uart_transaction	Sequence item: data byte, inject_parity_error, inject_framing_error, delay_before_send, o_fifo_full.
uart_driver	Drives UART frames bit-by-bit on i_rx_serial using the 16x baud tick protocol.
uart_monitor (RX)	Observes i_rx_serial and decodes frames sent to the DUT.
uart_monitor (TX)	Observes o_tx_serial and decodes frames emitted by the DUT.
uart_scoreboard	Compares RX and TX transactions. Classifies: clean match, parity-error match, framing drop, FIFO-full drop, mismatch.
uart_coverage_collector	3 covergroups: cg_protocol (data, error type), cg_fifo (flag states), cg_cross (error x FIFO state).
uart_master_seq	Orchestrates 5 test phases: directed, back-to-back, burst, error injection, random.
uart_if SVA (7 properties)	FIFO mutex, empty/full stability, overflow/underflow protection, TX handshake, TX busy glitch.

9. Known Limitations

- Single clock domain only. No CDC support between RX and TX (not required in loopback mode, but limits use in multi-clock systems).
- Baud rate error is not checked at elaboration. If CLK_FREQ is not divisible by (BAUD_RATE x 16), a silent fractional error occurs.

- FIFO depth must be a power of 2. The extra-bit pointer scheme does not support non-power-of-2 depths.
 - No flow control (RTS/CTS). Bytes are dropped silently when FIFO is full.
 - Single stop bit only. Extended stop bit configurations are not parameterized.
-

10. Revision History

Version	Description
1.0	Initial release. RTL design complete, UVM testbench with 7 SVA properties and functional coverage.